

Lenguajes de consulta

Son utilizados para operar con la base de datos.

Procedurales: instrucciones para realizar secuencia de operaciones “que y como”.

No Procedurales: solicita directamente la información deseada.

Algebra Relacional

Es un lenguaje de consultas procedimental.

Operaciones de una o dos relaciones (tablas) de entrada que generan una nueva relación (tabla) como resultado.

Operaciones fundamentales

- Unitarias (Selección, Proyección, Renombre)
- Binarias (Producto cartesiano, Unión, Diferencia)

Dadas las siguientes tablas:

Asociados=(idsocio, nombre, dirección, teléfono, sexo, estadocivil, fechanacimiento, idlocalidad)

Deportes=(iddeporte, nombre, monto_cuota, idsede)

Practica=(idsocio, iddeporte)

Localidad=(idlocalidad, nombre)

Sedes=(idsede, nombre, dirección, idlocalidad)

- **Selección** Operador σ .
Selecciona tuplas que satisfacen un predicado dado.
Ejemplo: mostrar todos los asociados casados.
 σ Estadocivil = “casado” (asociados)
- **Proyección** Operador π .
Devuelve la relación argumento con columnas omitidas.
Ejemplo: nombres de los asociados.
 π nombre(asociados)
- **Producto Cartesiano** Operador $\times$.
Conecta dos entidades de acuerdo a la definición matemática de la operación.
Ejemplo: mostrar cada asociado y la localidad donde vive.
 π apellido.nombre, localidad.nombre (σ asociado.idlocalidad = localidad.idlocalidad (asociado $\times$ localidad)

- **Renombrar** Operación ρ

Permite utilizar la misma tabla en, por ej., producto cartesiano.

Ejemplo: mostrar todos los asociados que viven en la misma dirección que el socio con id 75.

$$\pi_{\text{asociados.nombre}}(\sigma_{\text{Asociados.idDireccion}=\text{aso.idDireccion}}(\sigma_{\text{Asociados.idSocio}=75}(\text{Asociados}) \times \rho_{\text{asociados}}(\text{Asociados})))$$

- **Unión** Operador $\cup$

Tuplas comunes a dos relaciones, equivalente a la unión matemática. Debe efectuarse entre relaciones equivalentes (con mismo sentido/tipos).

Ejemplo: asociados que practiquen vóley o futbol.

$$\pi_{\text{asociados.nombre}}(\sigma_{\text{deportes.nombre}=\text{"futbol"}} \wedge \text{asociados.idSocio}=\text{practica.idSocio} \wedge \text{deportes.idDeporte}=\text{practica.idDeporte} (\text{Deporte X Practica X Asociados}))$$

$$\cup$$

$$\pi_{\text{asociados.nombre}}(\sigma_{\text{deportes.nombre}=\text{"vóley"}} \wedge \text{asociados.idSocio}=\text{practica.idSocio} \wedge \text{deportes.idDeporte}=\text{practica.idDeporte} (\text{Deporte X Practica X Asociados}))$$

- **Diferencia** Operador $-$

Diferencia entre dos relaciones, equivalente a la diferencia matemática. Debe efectuarse entre dos relaciones equivalentes (con mismo sentido/tipos).

Ejemplo: asociados que practiquen futbol y no vóley.

$$\pi_{\text{asociados.nombre}}(\sigma_{\text{deportes.nombre}=\text{"futbol"}} \wedge \text{asociados.idSocio}=\text{practica.idSocio} \wedge \text{deportes.idDeporte}=\text{practica.idDeporte} (\text{Deporte X Practica X Asociados}))$$

$$-$$

$$\pi_{\text{asociados.nombre}}(\sigma_{\text{deportes.nombre}=\text{"vóley"}} \wedge \text{asociados.idSocio}=\text{practica.idSocio} \wedge \text{deportes.idDeporte}=\text{practica.idDeporte} (\text{Deporte X Practica X Asociados}))$$

- **Producto Natural** Operador $|X|$

Hace el producto cartesiano con una selección de tuplas “con sentido” eliminando las columnas (atributos) repetidas. Si R y S dos relaciones no tienen atributos en común es igual al Producto Cartesiano.

Ejemplo: asociados que practican futbol.

$$\pi_{\text{asociados.nombre}}(\text{Asociados } |X| \text{ Practica } |X| \sigma_{\text{nombre}=\text{"futbol"}} (\text{Deportes}))$$

- **Intersección** Operador $\cap$

Tuplas que estén en dos relaciones, equivalente a la intersección matemática.
Debe efectuarse entre relaciones equivalentes (con mismo sentido/tipos).

Ejemplo: asociados que practiquen vóley y futbol.

$\pi_{\text{asociados.nombre}(\sigma_{\text{deportes.nombre}=\text{"futbol"} \wedge \text{asociados.idSocio}=\text{practica.idSocio} \wedge \text{deportes.idDeporte}=\text{practica.idDeporte}} (\text{Deporte X Practica X Asociados}))}$

$\cap$

$\pi_{\text{asociados.nombre}(\sigma_{\text{deportes.nombre}=\text{"vóley"} \wedge \text{asociados.idSocio}=\text{practica.idSocio} \wedge \text{deportes.idDeporte}=\text{practica.idDeporte}} (\text{Deporte X Practica X Asociados}))}$

- **Asignación** Operador $\leftarrow$

Expresión que asigna a una variable temporal el resultado de una operación.

Ejemplo: Temp $\leftarrow$ Operación del Algebra.

- **Producto** Operador $\bowtie$

$r[X] \bowtie s = \sigma_{\text{r[X] s}}$

- **Operaciones de Updates**

- Agregar tuplas: Deportes $\leftarrow$ Deportes $\cup$ ("golf", 5000, 21)
- Eliminar tuplas: Deportes $\leftarrow$ Deportes - ("bochas", 500, 1)
- Actualización de datos: δ saldo $\leftarrow$ saldo * (depósito)

Lenguaje de Consultas Estructurado – Structured Query Language (SQL)

Es un lenguaje declarativo usado para definir, consultar y manipular bases de datos relacionales. Es declarativo porque se especifica que se quiere obtener, no como se obtiene.

Modelo relacional

SQL se basa en el modelo relacional, donde la información se organiza en relaciones (tablas). Una tabla esta formada por tuplas (filas) y atributos (columnas).

Cada tabla representa una entidad o relación del mundo real.

Esquema vs instancia

- Esquema: estructura de la base de datos (tablas, atributos, restricciones).
- Instancia: datos actuales almacenados en un momento dado.

Keys

- Clave primaria (Primary Key (PK)): Identifica de forma única cada tupla.
- Clave foránea (Foreign Key (FK)): Referencia a la clave primaria de otra tabla.
- Clave candidata: Posibles claves únicas.
- Clave compuesta: Formada por mas de un atributo.

Restricciones de integridad

Reglas que garantizan consistencia:

- Integridad de entidad: la clave primaria no puede ser nula.
- Integridad referencial: las claves foráneas deben existir en la tabla referenciada.
- Restricciones de dominio: valores válidos para cada atributo.

Tipos de operaciones en SQL

DDL (Data Definition Language)

Define la estructura: crear, modificar o eliminar tablas.

Define el esquema.

DML (Data Manipulation Language)

Manipula datos: inserción, actualización, eliminación, consultas.

DCL (Data Control Language)

Control de premisos: autorización y roles.

TCL (Transaction Control Language)

Control de transacciones: commit, rollback, savepoint.

- **SELECT**
Recupera información de una Tabla.
SELECT a.nombre FROM Alumno a
- **WHERE**
Selecciona solo los registros que cumplen la condición (filtra filas).
SELECT a.nombre FROM Alumno a
WHERE a.nota > 6
- **AND/OR/NOT**
Combina condiciones:
AND -> ambas deben cumplirse.
OR -> al menos una se tiene que cumplir.
NOT -> niega condición.
- **IN**
Verifica si un valor está en una lista.
WHERE a.ciudad IN ('LP','BSAS')
- **BETWEEN**
Filtra valores entre dos límites.
WHERE a.edad BETWEEN 18 AND 30
- **LIKE**
Búsqueda por patrón, busca texto con comodines.
% -> varios caracteres.
_ -> un carácter.
WHERE a.nombre LIKE 'A%'
- **JOIN**
Combina tablas relacionadas, une datos de varias tablas usando una condición.
SELECT a.nombre, p.deporte
FROM Alumno a
JOIN Materia m ON (a.dni = m.dni)
- **UNION/INTERSECT/EXCEPT**
Las 3 necesitan que las consultas sean compatibles y eliminan repetidos.
UNION -> Une resultados (que estén en esta tabla o estén en esta).
INTERSECT -> Devuelve lo que esta en ambas (esta en una y esta en otra).
EXCEPT -> Devuelve lo que esta en la primero pero no en la segunda.
- **GROUP BY**
Agrupa filas iguales, permite aplicar funciones por grupo.
SELECT a.ciudad, COUNT(*) FROM Alumno a
GROUP BY ciudad

- **HAVING**
Filtrar grupos (como el WHERE pero para grupos).
SELECT a.ciudad, COUNT(*)
FROM Alumno a
GROUP BY Ciudad
HAVING COUNT(*) > 5
- **ORDER BY**
Ordena resultados ascendentemente o descendentemente.
SELECT * FROM Alumno
ORDER BY nombre ASC/DESC

FUNCIONES DE AGREGACION

- **COUNT**
Cuenta filas
SELECT COUNT(*) FROM Alumno
- **SUM**
Suma valores
SELECT SUM(a.faltas) FROM Alumno a
- **AVG**
Devuelve el promedio
SELECT AVG(a.nota) FROM Alumno a
- **MAX/MIN**
Devuelve el máximo y mínimo
SELECT MAX(a.edad), MIN (a.edad)
FROM Alumno a

ABM

- **INSERT**
Agregar datos. Inserta nuevas filas en una tabla.
INSERT INTO Table(dni, nombre) VALUES(4574, 'Demi')
- **DELETE**
Eliminar datos. Borra filas que cumplan una condición.
DELETE FROM Table WHERE Condicion
- **UPDATE**
Modifica datos. Cambia valores existentes.
UPDATE Alumno SET nombre = 'Demi' WHERE dni = 4574